데이터 베이스 개론 게시판만들기 보고서

20203067 서현우

국립창원대학교
CHANGWON NATIONAL UNIVERSITY

목차

0.실행예시

1.개요

2.코드 구조 및 역할

3.세부 코드 리뷰

index.js: 라우팅 담당

app.js: Express 애플리케이션 설정 및 시작

database.js: MySQL 데이터베이스 연결 설정

index.ejs: 메인페이지와 로그인 화면 구성

read.ejs: 게시글 상세조회 페이지 구성

register.ejs: 회원가입 페이지 구성

write.ejs: 게시글 작성 페이지 구성

개요

node.js기반의 express 프레임워크를 활용한 게시판과제에 대한 보고서입니다. 코드의 전체적인 구조와 각 부분의 역할, 데이터베이스와 jQuery의 동작방식을 설명하도록 하겠습니다. 우선 간략한 구조입니다.

코드 구조 및 역할

1. index.js : 라우팅 담당

메인 페이지 및 로그인 화면	사용자가 로그인했는지 여부에 따라 다른 내용을 렌더링
로그인 처리	사용자가 입력한 아이디와 비밀번호를 데이터베이스와 비교하여 세션에 사용자 정보를 저장
로그아웃 처리	세션을 파기하여 로그아웃을 처리
회원가입 페이지 이동 및 처리	회원가입 페이지로 이동하고, 회원가입 폼 제출 시 입력된 정보를 데이터베이스에 저장
아이디 중복 확인	회원가입 시 입력한 아이디의 중복 여부를 확인
게시글 목록 조회	게시글을 페이지 단위로 조회하고, 페이지네이션 기능을 지원
게시글 상세 조회	특정 게시글의 상세 내용을 조회하고, 조회수를 증가
댓글 작성 및 삭제	댓글을 작성하고 삭제하는 기능
게시글 작성 및 삭제	게시글을 작성하고 삭제하는 기능
좋아요 기능	좋아요 상태를 토글하고 게시글의 좋아요 수를 업데이트

2. app.js : Express 애플리케이션을 설정하고 시작하는 역할

Express 앱 설정	Express 애플리케이션을 생성하고 설정합니다.
바디 파서 미들웨어 사용	URL 인코딩 및 JSON 바디 파서를 사용합니다.
정적 파일 제공	public 디렉토리의 정적 파일을 제공하도록 설정합니다.
세션 설정	세션을 설정하여 사용자 로그인 상태를 관리합니다.
라우터 설정	index.js 라우터를 메인 라우터로 설정합니다.
서버 시작	지정된 포트에서 서버를 시작합니다.

3. database.js : MySQL 데이터베이스 연결을 설정합니다.

MySQL 데이터베이스 연결 설정	데이터베이스 연결을 위한 설정 정보를 제공합니다.
데이터베이스 연결	설정된 정보를 바탕으로 데이터베이스에 연결하고, 연결 실패 시 오류 메시지를 출력합니다.
모듈 내보내기	연결된 데이터베이스 객체를 모듈로 내보냅니다.

4. index.ejs : 메인페이지와 로그인 화면을 구성합니다.

로그인 화면	로그인 폼을 통해 사용자 로그인 처리
게시글 목록 조회	로그인 상태일 때 게시글 목록을 조회하고 페이지네이션 기능을 제공
jQuery	
로그인 폼 제출 처리	
페이지당 게시글 수 변경 처리	

5. read.ejs : 게시글 상세조회 페이지를 구성합니다.

게시글 상세 조회 페이지	게시글의 상세 내용을 조회하고, 댓글 목록 및 작성 기능을 제공
좋아요 기능	게시글에 대한 좋아요 버튼을 제공
게시글 삭제	게시글 작성자가 게시글을 삭제할 수 있는 기능
댓글 작성 및 삭제	댓글 작성 및 삭제 기능
jQuery	
좋아요 버튼 클릭 처리	
게시글 삭제 버튼 클릭 처리	
댓글 삭제 버튼 클릭 처리	
댓글 작성 폼 제출 처리	

6. register.ejs : 회원가입 페이지를 구성합니다.

회원가입 페이지	회원가입 폼을 통해 사용자 정보를 입력받아 회원가입 처리
아이디 중복 확인	입력한 아이디의 중복 여부를 확인
jQuery	
이메일 도메인 선택 처리	
아이디 중복 확인 버튼 클릭 처리	
회원가입 폼 제출 처리	

7. write.ejs : 게시글 작성 페이지를 구성합니다.

게시글 작성 페이지	게시글 작성 폼을 통해 새로운 게시글을 작성
jQuery	
게시글 작성 폼 제출 처리	

세부 코드 리뷰

코드와 함께 더 세부적으로 들여다 보겠습니다.

1. index.js: 라우팅 담당

메인 페이지 및 로그인 화면

기능 설명 : 사용자가 로그인했는지 여부에 따라 다른 내용을 렌더링합니다.

변수 설명 : req.session.username: 세션에 저장된 사용자 이름입니다.

동작 과정

클라이언트가 GET 요청을 보냅니다.

서버는 req.session.username을 확인합니다.

username이 세션에 존재하면 index.ejs 파일을 렌더링하며, username과 "환영합니다" 메시지를 전달합니다.

username이 세션에 존재하지 않으면 index.ejs 파일을 렌더링하며, "로그인" 메시지를 전달합니다.

```
// 메인 페이지 및 로그인 화면
router.get('/', (req, res) =>{
  if(req.session.username) {
                    res.render('index',        {        title:'환영합니다',
username:req.session.username});
else{
    res.render('index', { title:'로그인'});
}
});
```

로그인 처리

기능 설명 : 사용자가 입력한 아이디와 비밀번호를 데이터베이스와 비교하여 세션에 사용자 정보를 저장합니다.

변수 설명 :

req.body.username: 클라이언트가 입력한 사용자 이름입니다.

req.body.password: 클라이언트가 입력한 비밀번호입니다.

query: 사용자 검증을 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body에서 username과 password를 추출합니다.

username과 password를 이용해 데이터베이스에 쿼리를 실행합니다.

일치하는 사용자가 있으면 req.session.username에 username을 저장하고, 성공 응답을 반환합니다.

일치하는 사용자가 없으면 로그인 실패 응답을 반환합니다.

```
// 로그인 처리
router.post('/', (req, res) =>{
  constusername, password=req.body;
  const query =`SELECT * FROM user WHERE username = ? AND password = ?`;
  connection.query(query, [username, password], (err, results) =>{
    if(err) {
      console.error('database error:', err);
         return  res.status(500).json({  success:false,  message:'database
error'});
}
    if(results.length >0) {
      req.session.username =username; // 세션에 사용자 ID 저장
      console.log("로그인 성공");
      return res.status(200).json({ success:true});
} else{
      return res.status(401).json({ success:false, message:'로그인 실패'});
}
});
});
```

로그아웃 처리

기능 설명 : 세션을 파기하여 로그아웃을 처리합니다.

변수 설명 :

req.session.destroy: 세션을 파기하는 함수입니다.

동작 과정:

클라이언트가 GET 요청을 보냅니다.

서버는 req.session.destroy를 호출하여 세션을 삭제합니다.

세션 삭제에 성공하면 성공 메시지를 반환합니다.

세션 삭제에 실패하면 오류 메시지를 반환합니다.

```
// 로그아웃 처리
router.get('/logout', (req, res) =>{
  req.session.destroy((err) =>{
    if(err) {
       return res.status(500).json({ success:false, message:'로그아웃 실패
'});
}
    return res.status(200).json({ success:true});
});
});
```

회원가입 페이지 이동 및 처리

회원가입 페이지 이동

기능 설명 : 회원가입 페이지로 이동합니다.

동작 과정:

클라이언트가 GET 요청을 보냅니다.

서버는 register.ejs 파일을 렌더링합니다.

```
// 회원가입 페이지 이동
router.get('/register', (req, res) =>{
  res.render('register', { title:'회원가입'});
});
```

아이디 중복 확인 처리

기능 설명 : 회원가입 시 입력한 아이디의 중복 여부를 확인합니다.

변수 설명 :

req.body.username: 클라이언트가 입력한 사용자 이름입니다.

query: 아이디 중복 확인을 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body.username을 추출합니다.

username을 이용해 데이터베이스에 쿼리를 실행합니다.

결과가 0보다 크면 isDuplicate: true를 반환하고, 그렇지 않으면 isDuplicate: false를 반환합니다.

```
// 아이디 중복 확인 처리
router.post('/checkduplicateid', (req, res) =>{
  constusername=req.body;
  const query ="SELECT COUNT(*) AS count FROM user WHERE username = ?";
  connection.query(query, [username], (err, results) =>{
    if(err) {
      console.error('아이디 중복 확인 오류:', err);
        return  res.status(500).json({  isduplicate:false,  error:'database
error'});
}
    const isduplicate =results[0].count >0;
    res.json({ isduplicate});
});
});
```

회원가입 처리

기능 설명 : 회원가입 페이지에서 입력한 정보를 데이터베이스에 저장합니다.

변수 설명 :

req.body: 클라이언트가 입력한 회원가입 정보입니다.

query: 회원가입 정보를 저장하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body에서 회원가입 정보를 추출합니다.

모든 필드가 입력되었는지 확인하고, 비밀번호 확인이 일치하는지 검증합니다.

검증 통과 시 사용자 정보를 데이터베이스에 저장합니다.

저장 성공 시 index.ejs 파일을 렌더링하며 성공 메시지를 전달합니다.

저장 실패 시 오류 메시지를 반환합니다.

```
// 회원가입 처리
router.post('/register', (req, res) =>{
      constusername,     password,     passwordconfirm,     name,     email,
emaildomain=req.body;
  if(!username ||!password ||!name ||!email ||!emaildomain) {
    console.log("모든 필드를 입력해주세요.");
    return res.status(400).render('register', { title:'회원가입', error:'모
든 필드를 입력해주세요.'});
}
  if(password !==passwordconfirm) {
    console.log("비밀번호가 일치하지 않습니다.");
    return res.status(400).render('register', { title:'회원가입', error:'비
밀번호가 일치하지 않습니다.'});
}
   const  query  ="INSERT  INTO  user  (username,  password,  name,  email,
emaildomain) VALUES (?, ?, ?, ?, ?)";
  connection.query(query, [username, password, name, email, emaildomain],
(err) =>{
    if(err) {
      console.error('회원가입 오류:', err);
      return res.status(500).render('register', { title:'회원가입', error:'
회원가입 오류'});
}
    console.log("회원가입 성공");
     return res.render('index', { title:'회원가입 완료', message:'회원가입이
완료되었습니다. 잠시 후 로그인 페이지로 이동합니다.'});
});
});
```

게시글 목록 조회

기능 설명 : 게시글을 페이지 단위로 조회하고, 페이지네이션 기능을 지원합니다.

변수 설명 :

req.params.page: 클라이언트가 요청한 페이지 번호입니다.

req.query.limit: 페이지당 게시글 수입니다.

offset: SQL 쿼리에서 페이지 시작 지점을 계산하기 위한 변수입니다.

sql: 게시글 목록을 조회하기 위한 SQL 쿼리입니다.

countSql: 전체 게시글 수를 조회하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 GET 요청을 보냅니다.

서버는 req.params.page와 req.query.limit을 추출합니다.

page와 limit을 이용해 offset을 계산합니다.

데이터베이스에서 sql을 실행하여 해당 페이지의 게시글을 조회합니다.

countSql을 실행하여 총 게시글 수를 조회합니다.

총 페이지 수를 계산합니다.

조회된 게시글과 페이지 정보를 포함해 index.ejs 파일을 렌더링합니다.

```
// 게시글 목록 조회
router.get('/list', (req, res) =>{
```

```
  res.redirect('/list/1?limit=5'); // 기본적으로 5개 단위로 설정
});
router.get('/list/:page', (req, res) =>{
  const page =req.params.page;
  const limit =parseInt(req.query.limit) ||5; // 페이지당 게시글 수, 기본값
5
  const offset =(page -1) *limit;
  const sql ="SELECT postid, author, title, regdate, viewcnt, likes FROM
board ORDER BY postid DESC LIMIT ?, ?";
  connection.query(sql, [offset, limit], (err, rows) =>{
    if(err) {
      console.log("쿼리 실행 오류:", err);
      return res.status(500).send('게시글 목록 조회 오류');
}
    // 총 게시글 수 조회
    const countsql ="SELECT COUNT(*) AS count FROM board";
    connection.query(countsql, (counterr, countresult) =>{
      if(counterr) {
        console.log("게시글 수 조회 오류:", counterr);
        return res.status(500).send('게시글 수 조회 오류');
}
      const totalposts =countresult[0].count;
      const totalpages =Math.ceil(totalposts /limit);
      res.render('index', {
        title:'전체 글 조회',
        rows:rows,
        currentpage:parseInt(page),
        totalpages:totalpages,
        limit:limit, // 현재 페이지당 게시글 수
        username:req.session.username
});
});
});
```

게시글 상세 조회

기능 설명 : 특정 게시글의 상세 내용을 조회하고, 조회수를 증가시킵니다.

변수 설명 :

req.params.postid: 클라이언트가 요청한 게시글 ID입니다.

updateViewCountSql: 조회수를 증가시키기 위한 SQL 쿼리입니다.

sql: 게시글 상세 내용을 조회하기 위한 SQL 쿼리입니다.

replySql: 해당 게시글의 댓글을 조회하기 위한 SQL 쿼리입니다.

checkLikeSql: 좋아요 여부를 확인하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 GET 요청을 보냅니다.

서버는 req.params.postid를 추출합니다.

updateViewCountSql을 실행하여 조회수를 증가시킵니다.

sql을 실행하여 게시글 상세 내용을 조회합니다.

replySql을 실행하여 해당 게시글의 댓글을 조회합니다.
checkLikeSql을 실행하여 현재 사용자가 해당 게시글에 좋아요를 눌렀는지 확인합니다.
게시글 내용, 댓글, 좋아요 여부를 포함해 read.ejs 파일을 렌더링합니다.

```
// 게시글 상세 조회
router.get('/read/:postid', (req, res) =>{
  const postid =req.params.postid;
  // 조회수 증가 쿼리
  const updateviewcountsql ="UPDATE board SET viewcnt = viewcnt + 1 WHERE
postid = ?";
  connection.query(updateviewcountsql, [postid], (updateerr) =>{
    if(updateerr) {
      console.log("조회수 업데이트 오류:", updateerr);
}
    // 게시글 조회 쿼리
    const sql ="SELECT * FROM board WHERE postid=?";
    connection.query(sql, [postid], (err, row) =>{
      if(err) {
        console.log("쿼리 실행 오류:", err);
         return res.status(500).json({ success:false, message:'게시글 조회
오류'});
}
      const replysql ="SELECT * FROM reply WHERE postid=?";
      connection.query(replysql, [postid], (replyerr, replies) =>{
        if(replyerr) {
          console.log("댓글 조회 오류:", replyerr);
           return res.status(500).json({ success:false, message:'댓글 조회
오류'});
}
        // 좋아요 여부 확인
          const checkLikeSql ="SELECT * FROM likes WHERE postid = ? AND
username = ?";
           connection.query(checkLikeSql, [postid, req.session.username],
(likeErr, likeResults) =>{
          if(likeErr) {
            console.error('좋아요 확인 오류:', likeErr);
             return res.status(500).json({ success:false, message:'좋아요 확
인 오류'});
}
          const liked =likeResults.length >0;
              res.render('read', { title:'게시 글 상세 조회', row:row[0],
replies:replies, username:req.session.username, liked:liked});
});
});
});
});
```

댓글 작성 및 삭제

댓글 작성 기능

기능 설명 : 댓글을 작성하여 데이터베이스에 저장합니다.

변수 설명 :

req.body.postid: 클라이언트가 입력한 게시글 ID입니다.

req.body.commentWriter: 클라이언트가 입력한 댓글 작성자입니다.

req.body.commentContent: 클라이언트가 입력한 댓글 내용입니다.

sql: 댓글을 저장하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body에서 댓글 정보를 추출합니다.

sql을 실행하여 댓글을 데이터베이스에 저장합니다.

성공 시 해당 게시글 페이지로 리다이렉트하고, 실패 시 오류 메시지를 반환합니다.

```
// 댓글 작성 처리
router.post('/reply', (req, res) =>{
  constpostid, commentwriter, commentcontent=req.body;
   const sql ="INSERT INTO reply (postid, commentwriter, commentcontent,
commentdate) VALUES (?, ?, ?, NOW())";
  connection.query(sql, [postid, commentwriter, commentcontent], (err) =>{
    if(err) {
      console.log("댓글 작성 오류:", err);
       return res.status(500).json({ success:false, message:'댓글 작성 오류
'});
}
    res.redirect('/read/'+postid);
});
});
```

댓글 삭제 기능

기능 설명 : 사용자가 작성한 댓글을 삭제합니다.

변수 설명 :

req.body.commentid: 클라이언트가 요청한 댓글 ID입니다.

req.session.username: 세션에 저장된 사용자 이름입니다.

checkAuthorSql: 댓글 작성자를 확인하기 위한 SQL 쿼리입니다.

deleteReplySql: 댓글을 삭제하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body.commentid와 req.session.username을 추출합니다.

checkAuthorSql을 실행하여 댓글 작성자를 확인합니다.

작성자와 현재 사용자가 일치하면 deleteReplySql을 실행하여 댓글을 삭제합니다.

성공 시 성공 메시지를 반환하고, 실패 시 오류 메시지를 반환합니다.

```
// 댓글 삭제
router.post('/deletereply', (req, res) =>{
  const commentid =req.body.commentid;
  const username =req.session.username;
   const checkauthorsql ="SELECT commentwriter, postid FROM reply WHERE
```

```
commentid=?";
  connection.query(checkauthorsql, [commentid], (err, rows) =>{
    if(err) {
      console.log("작성자 확인 오류:", err);
       return res.status(500).json({ success:false, message:'작성자 확인 오
류'});
}
    if(rows.length >0 &&rows[0].commentwriter ===username) {
      connection.query("DELETE FROM reply WHERE commentid=?", [commentid],
(err) =>{
        if(err) {
          console.log("댓글 삭제 오류:", err);
           return res.status(500).json({ success:false, message:'댓글 삭제
오류'});
}
        res.status(200).json({ success:true});
} else{
      res.status(403).json({ success:false, message:'작성자만 삭제할 수 있습
니다.'});
}
});
});
```

게시글 작성 및 삭제

게시글 작성 기능

기능 설명 : 사용자가 작성한 게시글을 데이터베이스에 저장합니다.

변수 설명 :

req.body.title: 클라이언트가 입력한 게시글 제목입니다.

req.body.author: 클라이언트가 입력한 게시글 작성자입니다.

req.body.body: 클라이언트가 입력한 게시글 내용입니다.

sql: 게시글을 저장하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body에서 게시글 정보를 추출합니다.

sql을 실행하여 게시글을 데이터베이스에 저장합니다.

성공 시 게시글 목록 페이지로 리다이렉트하고, 실패 시 오류 메시지를 반환합니다.

```
// 글쓰기
router.post('/write', (req, res) =>{
  consttitle, author, body=req.body;
  const sql ="INSERT INTO board (author, title, body, regdate) VALUES (?,
?, ?, NOW())";
  connection.query(sql, [author, title, body], (err) =>{
    if(err) {
      console.log("쿼리 실행 오류:", err);
       return res.status(500).json({ success:false, message:'게시물 작성 오
류'});
```

```
}
    res.redirect('/list');
});
});
```

게시글 삭제 기능

기능 설명 : 사용자가 작성한 게시글을 삭제합니다.

변수 설명 :

req.body.postid: 클라이언트가 요청한 게시글 ID입니다.

req.session.username: 세션에 저장된 사용자 이름입니다.

checkAuthorSql: 게시글 작성자를 확인하기 위한 SQL 쿼리입니다.

deleteReplySql: 게시글에 달린 댓글을 삭제하기 위한 SQL 쿼리입니다.

deletePostSql: 게시글을 삭제하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body.postid와 req.session.username을 추출합니다.

checkAuthorSql을 실행하여 게시글 작성자를 확인합니다.

작성자와 현재 사용자가 일치하면 deleteReplySql을 실행하여 게시글에 달린 댓글을 삭제합니다.

이후 deletePostSql을 실행하여 게시글을 삭제합니다.

성공 시 성공 메시지를 반환하고, 실패 시 오류 메시지를 반환합니다.

```
// 게시글 삭제
router.post('/delete', (req, res) =>{
  const postid =req.body.postid;
  const username =req.session.username;
  const checkauthorsql ="SELECT author FROM board WHERE postid=?";
  connection.query(checkauthorsql, [postid], (err, rows) =>{
    if(err) {
      console.log("작성자 확인 오류:", err);
       return res.status(500).json({ success:false, message:'작성자 확인 오
류'});
}
    if(rows.length >0 &&rows[0].author ===username) {
             connection.query("DELETE   FROM   reply   WHERE   postid=?",
[parseInt(postid)], (err) =>{
        if(err) {
          console.log("댓글 삭제 오류:", err);
           return res.status(500).json({ success:false, message:'댓글 삭제
오류'});
}
               connection.query("DELETE   FROM   board   WHERE   postid=?",
[parseInt(postid)], (err) =>{
          if(err) {
            console.log("게시물 삭제 오류:", err);
             return res.status(500).json({ success:false, message:'게시물 삭
제 오류'});
}
          res.status(200).json({ success:true});
```

```
});
} else{
      res.status(403).json({ success:false, message:'작성자만 삭제할 수 있습니다.'});
}
});
});
```

좋아요 기능

좋아요 처리 기능

기능 설명 : 게시글에 좋아요를 추가하거나 취소합니다.

변수 설명 :

req.body.postid: 클라이언트가 요청한 게시글 ID입니다.

req.session.username: 세션에 저장된 사용자 이름입니다.

checkLikeSql: 좋아요 여부를 확인하기 위한 SQL 쿼리입니다.

insertLikeSql: 좋아요를 추가하기 위한 SQL 쿼리입니다.

deleteLikeSql: 좋아요를 취소하기 위한 SQL 쿼리입니다.

updateLikesSql: 게시글의 좋아요 수를 업데이트하기 위한 SQL 쿼리입니다.

getLikesSql: 게시글의 현재 좋아요 수를 조회하기 위한 SQL 쿼리입니다.

동작 과정:

클라이언트가 POST 요청을 보냅니다.

서버는 req.body.postid와 req.session.username을 추출합니다.

checkLikeSql을 실행하여 현재 사용자가 해당 게시글에 좋아요를 눌렀는지 확인합니다.

좋아요가 눌려있으면 deleteLikeSql을 실행하여 좋아요를 취소하고, updateLikesSql을 실행하여 좋아요 수를 감소시킵니다.

좋아요가 눌려있지 않으면 insertLikeSql을 실행하여 좋아요를 추가하고, updateLikesSql을 실행하여 좋아요 수를 증가시킵니다.

getLikesSql을 실행하여 현재 좋아요 수를 조회하고, 결과를 반환합니다.

```
// 좋아요 처리
router.post('/like', (req, res) =>{
  const postid =req.body.postid;
  const username =req.session.username;
  // 먼저 해당 사용자가 이 게시글을 좋아요 했는지 확인
  const checkLikeSql ="SELECT * FROM likes WHERE postid = ? AND username = ?";
  connection.query(checkLikeSql, [postid, username], (err, results) =>{
    if(err) {
      console.error('좋아요 확인 오류:', err);
       return res.status(500).json({ success:false, message:'좋아요 확인 오류'});
}
    if(results.length >0) {
      // 이미 좋아요를 한 경우, 좋아요 취소
        const deleteLikeSql ="DELETE FROM likes WHERE postid = ? AND username = ?";
```

```
      connection.query(deleteLikeSql, [postid, username], (err) =>{
        if(err) {
          console.error('좋아요 취소 오류:', err);
           return res.status(500).json({ success:false, message:'좋아요 취소
오류'});
}
        // 좋아요 수 감소 처리
         const updateLikesSql ="UPDATE board SET likes = likes - 1 WHERE
postid = ?";
        connection.query(updateLikesSql, [postid], (updateErr) =>{
          if(updateErr) {
            console.error('좋아요 수 감소 오류:', updateErr);
             return res.status(500).json({ success:false, message:'좋아요 수
감소 오류'});
}

          // 새로운 좋아요 수 가져오기
          const getLikesSql ="SELECT likes FROM board WHERE postid = ?";
          connection.query(getLikesSql, [postid], (getErr, getResults) =>{
            if(getErr) {
              console.error('좋아요 수 가져오기 오류:', getErr);
               return res.status(500).json({ success:false, message:'좋아요
수 가져오기 오류'});
}
                      res.status(200).json({   success:true,   liked:false,
likes:getResults[0].likes});
});
});
} else{
      // 좋아요를 누르지 않은 경우, 좋아요 등록
       const insertLikeSql ="INSERT INTO likes (postid, username) VALUES
(?, ?)";
      connection.query(insertLikeSql, [postid, username], (err) =>{
        if(err) {
          console.error('좋아요 등록 오류:', err);
           return res.status(500).json({ success:false, message:'좋아요 등록
오류'});
}
        // 좋아요 수 증가 처리
         const updateLikesSql ="UPDATE board SET likes = likes + 1 WHERE
postid = ?";
        connection.query(updateLikesSql, [postid], (updateErr) =>{
          if(updateErr) {
            console.error('좋아요 수 증가 오류:', updateErr);
             return res.status(500).json({ success:false, message:'좋아요 수
증가 오류'});
}
          // 새로운 좋아요 수 가져오기
          const getLikesSql ="SELECT likes FROM board WHERE postid = ?";
          connection.query(getLikesSql, [postid], (getErr, getResults) =>{
            if(getErr) {
              console.error('좋아요 수 가져오기 오류:', getErr);
```

```
                return res.status(500).json({ success:false, message:'좋아요
수 가져오기 오류'});
}
                        res.status(200).json({   success:true,   liked:true,
likes:getResults[0].likes});
});
});
}
});
});
```

2. app.js: Express 애플리케이션을 설정하고 시작하는 역할

Express 앱 설정

기능 설명 : Express 애플리케이션을 생성하고 설정합니다.

변수 설명 :

express(): Express 애플리케이션 인스턴스를 생성합니다.

app: 생성된 Express 애플리케이션 인스턴스를 저장합니다.

동작 과정:

express 모듈을 불러와 Express 애플리케이션 인스턴스를 생성합니다.

생성된 인스턴스를 app 변수에 저장합니다.

```
const express =require('express') // express 모듈 불러오기
const app =express() // express 앱 생성
```

바디 파서 미들웨어 사용

기능 설명 : URL 인코딩 및 JSON 바디 파서를 사용합니다.

변수 설명 :

bodyParser.urlencoded(): URL 인코딩된 데이터의 바디를 파싱합니다.

bodyParser.json(): JSON 형식의 데이터를 파싱합니다.

동작 과정:

bodyParser 모듈을 불러옵니다.

URL 인코딩된 데이터와 JSON 데이터를 파싱하는 미들웨어를 설정합니다.

```
const bodyParser =require('body-parser') // body-parser 모듈 불러오기
app.use(bodyParser.urlencoded({ extended:false})) // urlencoded 바디 파서
미들웨어 사용
app.use(bodyParser.json()) // JSON 바디 파서 미들웨어 사용
```

정적 파일 제공

기능 설명 : public 디렉토리의 정적 파일을 제공하도록 설정합니다.

변수 설명 :

express.static(): 정적 파일을 제공하는 미들웨어를 생성합니다.

path.join(): 파일 경로를 결합합니다.

동작 과정:

path 모듈을 불러옵니다.

public 디렉토리를 정적 파일 제공 디렉토리로 설정합니다.

```
const path =require('path') // path 모듈 불러오기
app.use(express.static(path.join(__dirname, 'public'))) // 정적 파일 제공
```

세션 설정

기능 설명 : 세션을 설정하여 사용자 로그인 상태를 관리합니다.

변수 설명 :

session(): 세션 미들웨어를 설정합니다.

secret: 세션 암호화 키입니다.

동작 과정:

express-session 모듈을 불러옵니다.

세션 설정을 통해 세션 암호화 키, 세션 재저장, 초기화 설정을 지정합니다.

```
app.use(session({
  secret:'asd123!',
  resave:false,
  saveUninitialized:false
}));
```

라우터 설정

기능 설명 : index.js 라우트를 메인 라우트로 설정합니다.

변수 설명 : router: index.js 파일에서 불러온 라우터입니다.

동작 과정:

index.js 파일에서 라우터를 불러옵니다.

불러온 라우터를 메인 라우트로 설정합니다.

```
const router =require('./routes/index');
app.use('/', router);
```

서버 시작

기능 설명 : 지정된 포트에서 서버를 시작합니다.

변수 설명 :

port: 서버가 청취할 포트 번호입니다.

app.listen(): 지정된 포트에서 서버를 시작합니다.

동작 과정:

포트 번호를 설정합니다.

지정된 포트에서 서버를 시작하고, 서버 시작 메시지를 출력합니다.

```
const port =3000 // 포트 번호 설정
app.listen(port, () =>{
  console.log(`server running at http://localhost:${port}/`);
});
```

3. database.js: MySQL 데이터베이스 연결을 설정합니다.

MySQL 데이터베이스 연결 설정

기능 설명 : 데이터베이스 연결을 위한 설정 정보를 제공합니다.

변수 설명 :

mysql.createConnection(): 데이터베이스 연결을 생성합니다.

db: 생성된 데이터베이스 연결 객체입니다.

동작 과정:

mysql 모듈을 불러옵니다.

데이터베이스 연결 설정을 지정하여 연결 객체를 생성합니다.

```
const mysql =require('mysql');
const connection =mysql.createConnection({
  host:'localhost',
  user:'root',
  password:'gusdn7447!',
  database:'boarddb'
});
```

데이터베이스 연결

기능 설명 : 설정된 정보를 바탕으로 데이터베이스에 연결하고, 연결 실패 시 오류 메시지를 출력합니다.

변수 설명 :

connection.connect(): 데이터베이스에 연결합니다.

동작 과정:

데이터베이스에 연결을 시도합니다.

연결 성공 시 성공 메시지를 출력하고, 실패 시 오류 메시지를 출력합니다.

```
connection.connect((err) =>{
  if(err) {
    console.error('database connection failed: '+err.stack);
    return;
}
  console.log('connected to database.');
});
```

모듈 내보내기

기능 설명 : 연결된 데이터베이스 객체를 모듈로 내보냅니다.

변수 설명 : module.exports: 모듈 내보내기를 설정합니다.

동작 과정:연결된 데이터베이스 객체를 다른 모듈에서 사용할 수 있도록 내보냅니다.

```
module.exports =connection;
```

4. index.ejs: 메인 페이지와 로그인 화면을 구성합니다.

로그인 화면

기능 설명: 로그인 폼을 통해 사용자 로그인 처리를 합니다.

변수 설명: username, password: 사용자가 입력한 아이디와 비밀번호입니다.

동작 과정:

사용자가 로그인 폼에 아이디와 비밀번호를 입력하고 제출합니다.

서버는 입력된 정보를 검증하고, 로그인 성공 시 세션에 사용자 정보를 저장합니다.

로그인 성공 시 게시글 목록 페이지로 이동합니다.

```
<form id="login-form">
    <input type="text"name="username"placeholder="아이디"required>
    <input type="password"name="password"placeholder="비밀번호"required>
    <input type="submit"value="로그인">
  </form>
  <script>
    $('#login-form').submit(function(event) {
      event.preventDefault();
      var username$('input[name="username"]').val();
      var password$('input[name="password"]').val();
      $.ajax({
        type:'POST',
        url:'/',
        contentType:'application/json',
        data:JSON.stringify({ username:username, password:password}),
        success:function(response) {
          window.location.href'/list/1?limit=5';
},
        error:function(xhr) {
          if(xhr.status=== 401) {
            alert('로그인 실패: 아이디 또는 비밀번호를 확인하세요.');
} else{
            alert('서버 오류');
}
}
});
  </script>
```

게시글 목록 조회

기능 설명: 로그인 상태일 때 게시글 목록을 조회하고 페이지네이션 기능을 제공합니다.

변수 설명:

rows: 데이터베이스에서 조회한 게시글 목록입니다.

currentPage, totalPages: 현재 페이지와 총 페이지 수입니다.

동작 과정:

사용자가 게시글 목록 페이지를 요청합니다.

서버는 데이터베이스에서 해당 페이지의 게시글을 조회합니다.

조회된 게시글 목록과 페이지 정보를 포함해 페이지를 렌더링합니다.

```
<table>
    <tr>
      <th>고유번호</th>
      <th>제목</th>
      <th>작성자</th>
      <th>작성일자</th>
      <th>조회수</th>
      <th>좋아요</th>
    </tr>
<% rows.forEach(function(row) { %>
      <tr>
        <td><%= row.postid %></td>
        <td><a href="/read/<%= row.postid %>"><%= row.title %></a></td>
        <td><%= row.author %></td>
              <td><%=  row.regdate.toLocaleString('en-US',  {  timeZone:
'Asia/Seoul' }) %></td>
        <td><%= row.viewcnt %></td>
        <td><%= row.likes %></td>
      </tr>
<% }); %>
  </table>
  <div class="pagination">
<% if (currentPage > 1) { %>
      <a href="/list/<%= currentPage - 1 %>?limit=<%= limit %>">이전</a>
<% } %>
<% for (let i = 1; i <= totalPages; i++) { %>
<% if (i === currentPage) { %>
        <strong><%= i %></strong>
<% } else { %>
        <a href="/list/<%= i %>?limit=<%= limit %>"><%= i %></a>
<% } %>
<% } %>
<% if (currentPage < totalPages) { %>
      <a href="/list/<%= currentPage + 1 %>?limit=<%= limit %>">다음</a>
<% } %>
  </div>
  <script>
    $('#limit').change(function() {
      var limit$(this).val();
      var currentPage'<%= currentPage %>';
      window.location.href'/list/'currentPage'?limit='limit;
});
    $('#logout-link').click(function(event) {
      event.preventDefault();
      $.ajax({
        type:'GET',
        url:'/logout',
        success:function() {
          window.location.href'/';
},
        error:function() {
```

```
        alert('로그아웃 실패');
}
});
  </script>
```

5. read.ejs: 게시글 상세 조회 페이지를 구성합니다.

게시글 상세 조회 페이지

기능 설명: 게시글의 상세 내용을 조회하고, 댓글 목록 및 작성 기능을 제공합니다.

변수 설명:

row: 조회된 게시글의 상세 정보입니다.

replies: 해당 게시글에 달린 댓글 목록입니다.

liked: 사용자가 해당 게시글에 좋아요를 눌렀는지 여부입니다.

동작 과정:

사용자가 특정 게시글을 선택합니다.

서버는 해당 게시글의 상세 내용을 조회하고, 조회수를 증가시킵니다.

댓글 목록을 조회합니다.

페이지를 렌더링하여 게시글 내용과 댓글을 표시합니다.

```
<table border="1">
    <tr>
      <td>번호</td>
      <td><%= row.postid %></td>
    </tr>
    <tr>
      <td>제목</td>
      <td><%= row.title %></td>
    </tr>
    <tr>
      <td>작성자</td>
      <td><%= row.author %></td>
    </tr>
    <tr>
      <td>작성일자</td>
      <td><%= row.regdate.toLocaleString('en-US', { timeZone: 'Asia/Seoul'
}) %></td>
    </tr>
    <tr>
      <td>내용</td>
      <td><%= row.body %></td>
    </tr>
    <tr>
      <td>좋아요</td>
      <td>
        <span id="like-count"><%= row.likes %></span>
          <button  id="like-button"><%=  liked  ?  '좋아요  취소'  :  '좋아요'
%></button>
      </td>
    </tr>
<% if (row.author === username) { %>
    <tr>
      <td colspan="2">
        <button id="delete-post-button">글 삭제</button>
```

```
        <a href="/list">목록</a>
      </td>
    </tr>
<% } else { %>
    <tr>
      <td colspan="2">
        <a href="/list">목록</a>
      </td>
    </tr>
<% } %>
  </table>
  <h2>댓글</h2>
  <table border="1">
<% replies.forEach(function(reply) { %>
    <tr>
      <td><%= reply.commentwriter %></td>
      <td><%= reply.commentcontent %></td>
          <td><%=  reply.commentdate.toLocaleString('en-US',  {  timeZone:
'Asia/Seoul' }) %></td>
      <td>
<% if (reply.commentwriter === username) { %>
                  <button    class="delete-reply-button"data-commentid="<%=
reply.commentid %>">댓글 삭제</button>
<% } %>
      </td>
    </tr>
<% }); %>
  </table>
  <h3>댓글 작성</h3>
  <form id="reply-form">
    <input type="hidden"name="postid"value="<%= row.postid %>">
    <table border="1">
      <tr>
        <td>작성자</td>
            <td><input  type="text"name="commentwriter"value="<%=  username
%>"readonly/></td>
      </tr>
      <tr>
        <td>내용</td>
                                                                <td><textarea
name="commentcontent"cols="30"rows="10"required></textarea></td>
      </tr>
      <tr>
        <td colspan="2">
          <button type="submit">댓글 작성</button>
        </td>
      </tr>
    </table>
  </form>
```

좋아요 기능

기능 설명: 게시글에 대한 좋아요 버튼을 제공합니다.

변수 설명:

postid: 게시글 ID입니다.

liked: 사용자가 좋아요를 눌렀는지 여부입니다.

동작 과정:

사용자가 좋아요 버튼을 클릭합니다.

서버에 좋아요 상태를 토글하는 요청을 보냅니다.

서버는 좋아요 상태를 업데이트하고, 현재 좋아요 수를 반환합니다.

반환된 좋아요 수와 상태를 페이지에 반영합니다.

```
<script>
    function likePost() {
        const postid$('input[name="postid"]').val();
        $.ajax({
            type:'POST',
            url:'/like',
            contentType:'application/json',
            data:JSON.stringify({ postid:postid}),
            success:function(response) {
                if(response.success) {
                    $('#like-count').text(response.likes);
                    if(response.liked) {
                        $('#like-button').text('좋아요 취소');
} else{
                        $('#like-button').text('좋아요');
}
} else{
                    alert(response.message);
}
},
            error:function(error) {
                console.error('좋아요 처리 오류:', error);
                alert('좋아요 처리 오류');
}
});
}
```

게시글 삭제

기능 설명: 게시글 작성자가 게시글을 삭제할 수 있는 기능입니다.

변수 설명:

postid: 삭제할 게시글 ID입니다.

동작 과정:

작성자가 게시글 삭제 버튼을 클릭합니다.

서버에 게시글 삭제 요청을 보냅니다.

서버는 게시글을 삭제하고 성공 메시지를 반환합니다.

성공 시 게시글 목록 페이지로 이동합니다.

```
    function deletePost() {
        const postid$('input[name="postid"]').val();
        $.ajax({
            type:'POST',
            url:'/delete',
            contentType:'application/json',
            data:JSON.stringify({ postid:postid}),
            success:function() {
                window.location.href'/list';
},
            error:function(error) {
                console.error('게시물 삭제 오류:', error);
                alert('게시물 삭제 오류');
}
});
}
```

댓글 작성 및 삭제

댓글 작성 기능

기능 설명: 댓글을 작성하여 데이터베이스에 저장합니다.

변수 설명:

postid, commentwriter, commentcontent: 댓글 작성에 필요한 정보입니다.

동작 과정:

사용자가 댓글 폼을 작성하고 제출합니다.

서버에 댓글 작성 요청을 보냅니다.

서버는 댓글을 저장하고 성공 메시지를 반환합니다.

페이지를 새로고침하여 댓글 목록을 업데이트합니다.

```
    $('#like-button').on('click', likePost);
    $('#delete-post-button').on('click', deletePost);
    $('.delete-reply-button').on('click', function() {
        const commentid$(this).data('commentid');
        deleteReply(commentid);
});
    $('#reply-form').on('submit', function(evt) {
        evt.preventDefault();

        const formData$(this).serialize();
        $.ajax({
            type:'POST',
            url:'/reply',
            data:formData,
            success:function() {
                location.reload();
},
            error:function(error) {
                console.error('댓글 작성 오류:', error);
                alert('댓글 작성 오류');
```

```
}
});
});
```

댓글 삭제 기능

기능 설명: 사용자가 작성한 댓글을 삭제합니다.

변수 설명:

commentid: 삭제할 댓글 ID입니다.

동작 과정:

사용자가 댓글 삭제 버튼을 클릭합니다.

서버에 댓글 삭제 요청을 보냅니다.

서버는 댓글을 삭제하고 성공 메시지를 반환합니다.

페이지를 새로고침하여 댓글 목록을 업데이트합니다.

```
    function deleteReply(commentid) {
        $.ajax({
            type:'POST',
            url:'/deletereply',
            contentType:'application/json',
            data:JSON.stringify({ commentid:commentid}),
            success:function() {
                location.reload();
},
            error:function(error) {
                console.error('댓글 삭제 오류:', error);
                alert('댓글 삭제 오류');
}
});
}
```

6. register.ejs: 회원가입 페이지를 구성합니다.

회원가입 페이지

기능 설명: 회원가입 폼을 통해 사용자 정보를 입력받아 회원가입을 처리합니다.

변수 설명:

username, password, passwordconfirm, name, email, emaildomain: 회원가입에 필요한 정보입니다.

동작 과정:

사용자가 회원가입 폼에 정보를 입력하고 제출합니다.

서버는 입력된 정보를 검증하고, 유효성 검사를 통과하면 데이터베이스에 저장합니다.

회원가입 성공 시 로그인 페이지로 이동합니다.

```
<form id="register-form">
  <h1>회원가입</h1>
<% if (error) { %>
    <div class="error-message"><%= error %></div>
<% } else if (message) { %>
    <div class="success-message"><%= message %></div>
<% } %>
  <p>
    <label>이름
      <input type="text"name="name"placeholder="이름 입력"value="<%= name ?
name : '' %>">
    </label>
  </p>
  <p>
    <label>아이디
       <input type="text"name="username"placeholder="아이디 입력"value="<%=
username ? username : '' %>">
         <button  type="button"id="check-duplicate-button">아이디  중복확인
</button>
    </label>
  </p>
  <p>
    <label>비밀번호
      <input type="password"name="password"placeholder="비밀번호 입력">
    </label>
  </p>
  <p>
    <label>비밀번호 확인
       <input type="password"name="passwordconfirm"placeholder="비밀번호 재
입력">
    </label>
  </p>
  <p>
    <label>이메일
      <input type="text"name="email"value="<%= email ? email : '' %>">@
          <input  type="text"name="emaildomain"id="emaildomain"value="<%=
emaildomain ? emaildomain : '' %>">
      <select id="email_sel">
```

```
        <option value="">직접 입력</option>
        <option value="naver.com">NAVER</option>
        <option value="hanmail.net">DAUM</option>
        <option value="gmail.com">GOOGLE</option>
      </select>
    </label>
  </p>
  <input type="submit"value="회원가입">
</form>
<script>
  $('#email_sel').change(function() {
    var emaildomain$(this).val();
    $('#emaildomain').val(emaildomain);
});
  $('#check-duplicate-button').click(function() {
    var username$("input[name='username']").val();
    if(!username) {
      alert("아이디를 입력해주세요.");
      return;
}
    $.ajax({
      type:'POST',
      url:'/checkduplicateid',
      contentType:'application/json',
      data:JSON.stringify({ username:username}),
      success:function(response) {
        if(response.isDuplicate) {
          alert("아이디가 중복되었습니다. 다른 아이디를 입력해주세요.");
} else{
          alert("사용 가능한 아이디입니다.");
}
},
      error:function(xhr) {
        alert('아이디 중복 확인 오류');
}
});
});
  $('#register-form').submit(function(event) {
    event.preventDefault();
    var formData$(this).serialize();
    $.ajax({
      type:'POST',
      url:'/register',
      data:formData,
      success:function(response) {
        window.location.href'/';
},
      error:function(xhr) {
        if(xhr.responseJSON && xhr.responseJSON.message) {
          alert(xhr.responseJSON.message);
} else{
          alert('회원가입 중 오류가 발생했습니다. 다시 시도해 주세요.');
```

```
}
}
});
});
</script>
```

7. write.ejs: 게시글 작성 페이지를 구성합니다.

게시글 작성 페이지

기능 설명: 게시글 작성 폼을 통해 새로운 게시글을 작성합니다.

변수 설명:

title: 게시글 제목

author: 게시글 작성자

body: 게시글 내용

동작 과정:

사용자가 게시글 작성 폼에 정보를 입력하고 제출합니다.

서버는 입력된 정보를 데이터베이스에 저장합니다.

저장 성공 시 게시글 목록 페이지로 이동합니다.

```
<form id="write-form">
    <table border="1">
      <tr>
        <td>제목</td>
        <td><input type="text"name="title"required></td>
      </tr>
      <tr>
        <td>작성자</td>
                <td><input   type="text"name="author"value="<%=   username
%>"readonly></td>
      </tr>
      <tr>
        <td>내용</td>
                                                                <td><textarea
name="body"cols="30"rows="10"required></textarea></td>
      </tr>
      <tr>
        <td colspan="2">
          <button type="submit">작성</button>
        </td>
      </tr>
    </table>
  </form>
  <script>
    $('#write-form').submit(function(event) {
      event.preventDefault();
      var formData$(this).serialize();
      $.ajax({
        type:'POST',
        url:'/write',
        data:formData,
        success:function() {
          window.location.href'/list';
},
        error:function(error) {
          alert('게시물 작성 오류');
```

```
}
});
  </script>
```

결론

본 보고서에서는 Node.js 기반의 Express 프레임워크와 MySQL 데이터베이스를 활용하여 게시판 시스템을 구현하는 과정을 다루었습니다. 각 파일의 역할과 코드 동작 과정을 상세히 설명하며, 게시글 작성, 조회, 수정, 삭제 등의 기본 기능을 포함한 전체적인 흐름을 이해할 수 있도록 구성하였습니다. 이를 통해 게시판 시스템을 구축하고 유지보수하는 데 필요한 기본적인 기술과 개념을 습득할 수 있었습니다.